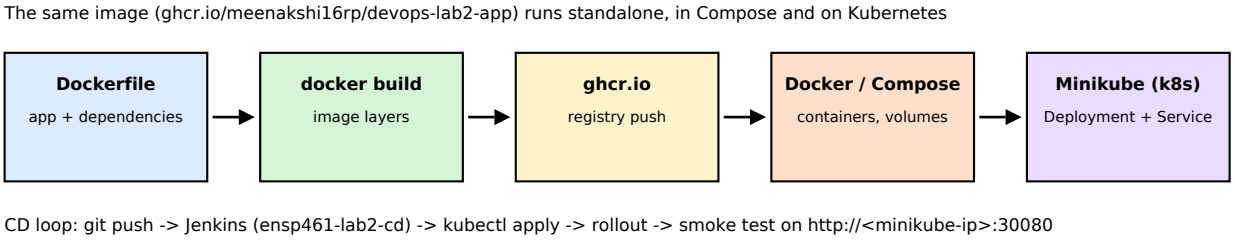
Question 9: Prepare a containerization and orchestration report describing the architecture, deployment strategy, scaling results, and observations with screenshots.

Executed on Ubuntu 24.04 LTS (WSL2) by Meenakshi Rakesh. Repository: https://github.com/meenakshi16rp/Devops-lab-2

1. Architecture



- Build: the Flask app is packaged by a layer-cached Dockerfile into ghcr.io/meenakshi16rp/devops-lab2-app and pushed to the registry.
- Run (Docker/Compose): standalone container on port 5001; Compose adds a redis service for the visit counter on port 5002.
- Orchestrate (Minikube): Kubernetes v1.37.0 single-node cluster; a ConfigMap injects the greeting, the Deployment runs 2 replicas pulled from ghcr via imagePullSecrets, and a NodePort Service exposes them on 30080.
- CD: the Jenkins job ensp461-lab2-cd applies the manifests and smoke-tests the service on every git push.

2. Tools and environment

- Ubuntu 24.04 LTS (WSL2) as user meenakshi; Docker Engine 29.9.2, Docker Compose v5.7.0, kubectl v1.37.2, Minikube v1.39.0 (Kubernetes v1.37.0, docker driver).
- Registry: ghcr.io/meenakshi16rp (GitHub PAT authentication, imagePullSecrets for the cluster).
- CI/CD: Jenkins 2.580.2 LTS - job ensp461-lab2-cd, Pipeline script from SCM, Poll SCM H/5 * * * *.

3. Deployment strategy

- Rolling update (default Deployment strategy): kubectl set image to 2.1.0 created a new ReplicaSet; new pods became Ready before old pods terminated, so the NodePort service kept answering during the update.
- Rollback: kubectl rollout undo restored revision 1 (image 2.0.0) and the service immediately served the previous version again.

- Scaling: kubectl scale --replicas=4 / --replicas=2 adjusted the number of pods without any downtime.

4. Scaling results

Command	Replicas	Observed pods
initial deploy	2	2/2 Running
kubectl scale --replicas=4	4	4 Running (2 added, rollout successful)
kubectl rollout undo	4 -> 2	revision 1 image 2.0.0 served again
kubectl scale --replicas=2	2	2 Running, extras terminated gracefully

5. Results

Operation	Command	Outcome
Image build	docker build -t ghcr.io/...:2.0.0	207 MB image, layers cached
Registry push	docker push (:2.0.0, :latest)	digest sha256:79a66362...
Compose stack	docker compose up -d	web + redis, visits 1 -> 2 via redis
Layer loss	docker rm + fresh container	/app/data.txt gone (ephemeral)
Named volume	-v lab2-data:/data	note.txt survived container deletion
Bind mount	-v ~/bindmount-demo:/hostdata	host <-> container file exchange
K8s deploy	kubectl apply -f k8s/	2/2 pods Running, NodePort 30080 UP
Rolling update	kubectl set image ...:2.1.0	zero downtime, new ReplicaSet
Rollback	kubectl rollout undo	version 2.0.0 served again
Scaling	kubectl scale --replicas=4	4 pods Running, then back to 2
Continuous deployment	git push -> Jenkins CD	build #4 by SCM change, SUCCESS

6. Evidence highlights

```
meenakshi@ubuntu: ~/ensp461-lab2 (kubectl get pods)

NAME                        READY   STATUS    RESTARTS   AGE   IP           NODE       \
      NOMINATED NODE   READINESS GATES
lab2-app-59d84c84c4-qp9pp   1/1     Running   0          54s   10.244.0.2   minikube   \
      <none>           <none>
lab2-app-59d84c84c4-szv2z   1/1     Running   0          54s   10.244.0.4   minikube   \
      <none>           <none>

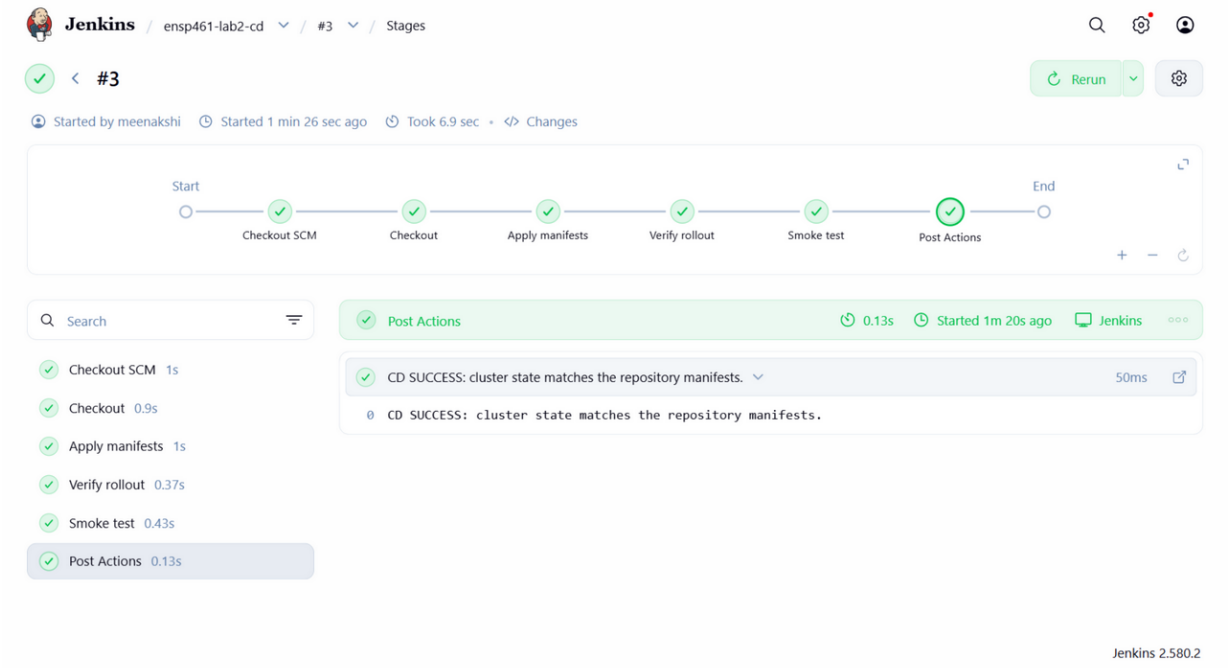
meenakshi@ubuntu:~$ kubectl get deployment,svc
NAME                        READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/lab2-app    2/2     2            2           55s

NAME                 TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
service/kubernetes   ClusterIP   10.96.0.1        <none>        443/TCP          75s
service/lab2-svc     NodePort    10.103.154.210   <none>        5000:30080/TCP   55s

meenakshi@ubuntu:~$ kubectl describe pod -l app=lab2-app | grep -E \
      "Image:|State:|Started:" | head -6
    Image:          ghcr.io/meenakshi16rp/devops-lab2-app:2.0.0
    State:          Running
      Started:      Wed, 04 Nov 2026 20:04:45 +0000
    Image:          ghcr.io/meenakshi16rp/devops-lab2-app:2.0.0
    State:          Running
      Started:      Wed, 04 Nov 2026 20:04:43 +0000

meenakshi@ubuntu:~$ curl -s http://192.168.49.2:30080/   (via NodePort 30080)
{"hostname":"lab2-app-59d84c84c4-szv2z","message":"Hello from Meenakshi Kubernetes \
      Lab","version":"2.0.0","visits":0}


meenakshi@ubuntu:~$ curl -s http://192.168.49.2:30080/   (repeat - different pod)
{"hostname":"lab2-app-59d84c84c4-szv2z","message":"Hello from Meenakshi Kubernetes \
      Lab","version":"2.0.0","visits":0}
```



7. Observations

- One image, three runtimes: the identical ghcr image ran as a standalone container, inside Compose and on Kubernetes - containerization decoupled the app from its runtime.
- Declarative manifests + kubectl apply make cluster state reproducible; the CD job simply re-applies the repository on every push.
- Defects found during the lab were all caught by evidence: missing app.run() (exit 0 with empty logs), missing jsonify import (HTTP 500), jenkins-user minikube CLI limitation and the removed .InternalIP field in Kubernetes 1.37 - each fixed and re-verified through the same pipeline.
- Minikube's docker driver keeps the whole lab inside the authorized Ubuntu (WSL2) environment; no resources outside the student identity (meenakshi) were created.

8. Conclusion

The sample application is fully containerized (Dockerfile, 207 MB image, pushed to ghcr.io/meenakshi16rp), runs as a multi-container stack under Docker Compose, and is orchestrated on a Minikube Kubernetes cluster with a ConfigMap-injected Deployment, a NodePort Service, rolling updates, rollback and scaling. Continuous deployment is automated end to end: a git push to meenakshi16rp/Devops-lab-2 triggers the Jenkins job ensp461-lab2-cd, which applies the manifests and smoke-tests the service. All evidence (command logs, terminal captures, screenshots and this report) is committed under docs/ in the repository.